COWBOY ·WHITEPAPER SERIES

Cowboy: Design Decisions Overview

STATUS	TYPE	CATEGORY
Draft for internal review	Standards Track	Core
AUTHOR(S)	**CREATED**	**UPDATED**
Cowboy Foundation	2025-09-17	2026-08-24
LICENSE		
CC0-1.0		

> **Note:** This document explains the "why" behind Cowboy's architecture. For complete technical specifications, see the Technical Whitepaper, the Storage Whitepaper, the Secrets Whitepaper, and the CBQS Whitepaper. This Markdown file is authoritative; the committed PDF is a point-in-time rendering of it.

Abstract

Cowboy is the open operating system for autonomous software. Its consensus primitive is the **actor**: a durable Python program with identity, actor-scoped state, a mailbox, a balance, and a protocol-native clock. Actors can hold assets, exchange messages, schedule future work, and pay for external capabilities without depending on an off-chain keeper. This gives autonomous software a path to become a native citizen of a digital economy rather than a tenant behind a platform account.

Cowboy combines a **Python-based actor-model execution environment** with **proof-of-stake consensus** and protocol-mediated off-chain computation. For heavy tasks — LLM inference, web requests, MCP tool calls, or containerized jobs — **Runners** execute work under explicit verification, authorization, and payment rules. A selected Runner is part of the trust boundary unless the job requests and receives stronger isolation; off-chain execution is not uniformly confidential or cryptographically proven.

Applications decide which state transitions need public consensus and which work belongs on provider data paths. Cowboy's architecture includes **CBFS** for private and public volumes, **CBSS** for threshold-authorized secret and key release, **CBQS** for durable workload coordination, and the Runner market for compute. CBFS, CBSS, and CBQS form Cowboy's **sovereign substrate**: near-chain services whose required authorization, commitments, or settlement records are anchored on Cowboy while high-volume data paths remain off the replicated state. Session payments, routing, event subscriptions, and the Trading Post add application distribution and commerce without creating a privileged platform contract.

To ensure fair and predictable resource pricing, Cowboy introduces a **dual-metered gas model**, separating pricing for computation (**Cycles**) and data (**Cells**) into independent, EIP-1559-style fee markets. Security is provided by **Simplex BFT** consensus with **proof-of-stake**, fast finality, and VRF-based leader election. Python makes this execution model accessible to both human developers and AI coding systems without turning every workload into consensus execution.

Introduction

The Problem: A Chasm Between Intelligence and Agency

Most programmable chains evolved from a synchronous, function-call model that couples state, compute, and timing inside a transaction. This model is ill-suited to software that waits, schedules itself, calls models and web services, and coordinates across independent workers. It also pushes developers into specialized languages even though most AI and enterprise systems are built in Python.

The usual workaround is a collection of smart contracts, cloud servers, cron jobs, data feeds, object stores, secret vaults, message brokers, gateways, and payment processors. Each component has its own identity, authorization, and billing model. There is no shared rule that binds a piece of off-chain work to a particular account, actor, job, capability grant, or settlement.

There is also an ownership failure: most AI applications are **tenants**, not citizens. Their models, compute, data, and credentials live in accounts controlled by platform operators. A system assembled this way can be deplatformed, repriced, or inspected by the parties it rents from. Intelligence without a durable ownership and authority layer is not agency.

Not every byte belongs on-chain. Consensus state is public and scarce; model weights, documents, credentials, prompts, and high-frequency coordination usually are not. The architectural problem is therefore not how to force an entire application into consensus. It is how to make the boundary between replicated state and off-chain providers explicit, capability-scoped, economically accountable, and inspectable.

The Solution: Cowboy

Cowboy makes the actor the durable identity and control point. The chain delivers messages and timers, enforces resource and economic rules, and commits the state transitions that require consensus. Runners and provider networks supply compute, storage, secret release, workload coordination, and ingress under subsystem-specific trust profiles. Applications compose these pieces per boundary instead of inheriting one blanket claim about privacy, verification, or provider independence.

This document explains the architectural decisions and trade-offs that shape Cowboy's design.

Key Innovations in Cowboy

General-purpose chains struggle with asynchronous, data-hungry, latency-sensitive software. Cowboy addresses that through six technical choices:

- **Deterministic Python Actors:** A sandboxed Python VM with mailbox messaging, reentrancy (depth-capped), and first-class support for the world's most popular programming language.
- **Native Timers & Scheduler:** Protocol-native scheduling combines per-fire fee payment, a timer-lane EIP-1559 basefee, priority tips, and bounded per-actor fairness without requiring an external keeper network.
- **Protocol-Mediated Off-Chain Compute:** A Runner market for LLM inference, HTTP fetches, MCP tool calls, custom compute, and one-shot container jobs, with explicit trust profiles rather than one universal verification claim.
- **A Sovereign Substrate of Near-Chain Services: CBFS**, **CBSS**, and **CBQS** keep bulk and private payloads off replicated state while using scoped capabilities, registries, commitments, and settlement at the boundaries where they are required.
- **Application and Commerce Rails:** Session payments, routing, event subscriptions, container policy, and the Trading Post make software addressable, hireable, and payable through protocol-defined interfaces.
- **Dual-Metered Gas:** Independent pricing and EIP-1559-style fee markets for compute (**Cycles**) and data/storage (**Cells**) to ensure fair and predictable costs.

Two Ways to Build

Cowboy supports two named application shapes. A **consensus application** centers deterministic actor execution and replicated state. A **sovereign application** pairs chain-anchored ownership and authority with predominantly provider-served execution and data. These are useful poles, not a mechanical classification rule: execution locality alone does not establish sovereignty, and most applications combine both shapes.

Consensus Applications

In a consensus application, the actor's on-chain state and deterministic Python handlers are the center of the application. Ownership changes, balances, schedules, policies, and consequential decisions are public and independently reproducible. A trading agent, DeFi rebalancer, oracle committee, or on-chain game may call a Runner for an LLM analysis or web result, but that off-chain work is an input to a state transition committed by consensus.

Everything in Cowboy's consensus core — deterministic Python execution, single-block atomicity, native timers, dual-metered gas, and VRF ordering — serves this pole.

Sovereign Applications

In a sovereign application, the system is **chain-anchored and Runner-centric**: its durable identity, ownership, authorization, commitments, and settlement live on Cowboy, while most executable logic runs in Runner jobs or persistent workloads. Private and high-volume data use near-chain services operated by storage relays, a secret-release proxy committee, and queue brokers. A hosted workspace, personal AI with years of memory, or team of cooperating agents is mostly data plane: documents and embeddings, external credentials, long-running workloads, and rapid worker-to-worker traffic do not belong in replicated consensus state.

For this pole, the visible actor is the tip of an iceberg. Below the waterline sit CBFS volumes holding application data, CBSS policies guarding secrets and keys, CBQS streams coordinating workloads, and Runners supplying execution. The chain is the **authority record**, not the bulk data plane: it determines who owns a resource, what a capability authorizes, which work may receive a secret, which commitments or receipts were posted, and how payment settles.

We call this shape *sovereign* because its ownership stack terminates at the user rather than a platform landlord. Sovereignty is about who owns and controls the application, not simply where its code runs. It is a subsystem-specific design claim, not a promise that every provider is blind or interchangeable. A selected Runner can see the plaintext supplied to its job unless a stronger isolation mechanism applies; public CBFS volumes are intentionally readable; and CBQS brokers retain operational metadata and authoritative consumer state. Provider replacement therefore requires an explicit subsystem procedure for export, continuity, reassignment, recovery, and revocation rather than an assumption of seamless migration.

Most applications use both shapes. A hireable trading agent can keep commerce, policy, and settlement in consensus while storing private memory in CBFS, releasing credentials through CBSS, coordinating workers through CBQS, and running model or container code on selected Runners. Cowboy supplies one chain-anchored identity and settlement layer across the composition; each near-chain provider may still be authoritative for the operational state its subsystem assigns to it.

Why Python?

Python is the language of AI and the glue of modern software. Cowboy uses its syntax, data model, and developer familiarity for actor code while imposing the restrictions required by replicated execution. The PVM is a deterministic, RustPython-derived runtime rather than CPython: it excludes C extensions, networking, threads, ambient filesystem access, dynamic imports, and other host-dependent behavior. Libraries that depend on native extensions — including most model-training and numerical-computing stacks — run through Runners rather than inside consensus.

The trade-off is deliberate. Actor authors get a familiar language and can share schemas, pure-Python utilities, tests, and application logic with ordinary Python systems; they do not get drop-in compatibility with every PyPI package. Cowboy's claim is therefore **deterministic actor development using a restricted Python language surface**, paired with Runner access to the broader Python ecosystem when work does not belong in consensus.

The Actor Model

Cowboy's core execution model is based on the actor model, a paradigm that naturally fits autonomous, asynchronous systems.

Why Actors?

The actor model provides several key advantages for autonomous agents:

1. **Natural Asynchrony:** Actors communicate via asynchronous messages, matching the real-world behavior of autonomous systems that interact with external services, wait for responses, and operate on independent schedules.
2. **Encapsulation:** Each actor has actor-scoped state that can only be modified through its handlers. This isolates mutation between actors, but it is not a privacy claim: validators replicate and execute consensus state.
3. **Composability:** Complex systems emerge from simple actors sending messages to each other. This modularity makes it easier to reason about, test, and audit autonomous systems.
4. **Autonomy:** Actors can schedule their own execution via timers, enabling true autonomy without external keeper networks.

Single-Block Atomicity

Cowboy provides **atomicity only within one handler execution in a single block**. State changes and handler-created outbound messages, timers, deferred transactions, and events participate in the same nested rollback domain: they commit or revert together. There is no cross-block atomicity: providing it would require either global locks (destroying parallelism and creating deadlock vectors) or speculative execution with rollbacks (creating griefing opportunities and unpredictable costs). Cowboy explicitly rejects these approaches.

The consequence for developers: when a handler waits on an off-chain result, the continuation runs later, against potentially different world state, and must re-validate its assumptions. Cowboy makes this boundary explicit rather than hiding it.

Awaiting Off-Chain Work

The SDK surfaces the boundary as ordinary `async` / `await`. A handler that needs off-chain work awaits it; the runtime suspends the handler, the runner executes, and the continuation resumes in a later block as its own atomic transaction. Context that must survive the gap is captured explicitly. From `examples/core/09-runner-llm` (trimmed):

```python
from cowboy_sdk import actor, runner, capture, codec, runtime

@actor
class RunnerLLMActor:

    @runner.continuation
    async def ask_llm(self, payload):
        obj = codec.decode(payload) if payload else {}
        prompt = str(obj.get("prompt", ""))
        request_id = self.storage.get("request_count", 0)
        self.storage["request_count"] = request_id + 1
        self.storage[f"llm:{request_id}:status"] = "pending"

        ctx = capture()              # context that must cross the block boundary
        ctx.request_id = request_id

        result = await runner.llm(   # suspends; resumes in a later block
            prompt,
            system_prompt=SYSTEM_PROMPT,
            max_tokens=256,
        )

        request_id = ctx.request_id  # restored from captured context
        self.storage[f"llm:{request_id}:response"] = _extract_llm_response(result)
        self.storage[f"llm:{request_id}:status"] = "done"
        runtime.emit_event("llm.response", {"request_id": request_id})
```

Three principles hide in this small example:

- **No hidden control flow.** The `await` desugars to explicit message passing — a job message out, a result message back in, each its own on-chain transaction. Nothing is a closure held in memory; execution can be traced block by block.
- **Explicit context capture.** Only what is written to `capture()` (or storage) crosses the boundary. There is no accidental closure over stale local variables — what survives the gap is a deliberate, visible choice.
- **Re-validation is the developer's job.** The continuation runs against current state. Code that acted on pre-await reads (a price, a balance) must re-read and re-check them — the model makes the staleness possible to see, and the pattern makes checking it natural.

The Runner system is not special syntax: it is a system actor that receives job requests and sends results. The same message-passing substrate carries actor-to-actor calls and timer callbacks.

Native Timers and Scheduling

Cowboy provides protocol-native timers and scheduling. Actors can schedule messages to themselves or other actors at a future block height or on a recurring interval without operating a separate keeper integration. Firing remains subject to timer-lane capacity, fee-payer balance, and expiry.

Why Protocol-Level Timers?

External keeper networks (like Ethereum's Gelato or Chainlink Automation) introduce several problems:

1. **Centralization Risk:** Keepers are typically operated by a small number of entities, creating single points of failure.
2. **Cost Inefficiency:** Each keeper network requires separate infrastructure, payment mechanisms, and trust assumptions.
3. **Coordination Overhead:** Actors must integrate with external services, manage subscriptions, and handle keeper failures.
4. **MEV (Maximal Extractable Value) Exposure:** Keepers can observe scheduled transactions and potentially front-run them.

By making timers native to the protocol, Cowboy moves scheduling admission, payment, ordering, and failure handling into one consensus-visible mechanism. It removes a separate keeper integration and its operator-selection problem; it does not guarantee that every due timer fires in its target block under congestion.

Scalable Design: Height-Indexed Storage

The scheduler stores timers indexed by their target block `height`, with the same-height bucket ordered FIFO by insertion. This keeps the per-block work proportional to the number of timers actually firing this block, not to the total active population, and avoids the operational complexity of a priority queue or tiered calendar.

A separate `LANE_TIMER_CYCLES` budget (**8,800,000 cycles**, 11% of the 80M block cap; 22% of the 40M non-system lane total) is dedicated to timer execution, and a parallel `TIMER_GC_CYCLES` budget handles TTL-expiry cleanup so a sweep storm cannot starve live execution. A per-actor cap (`max_timers_per_actor = 1,024`) bounds individual exposure.

Economic Rationality: The Per-Fire Fee Payer Model

Each timer records `fee_payer`, `gas_limit_per_fire`, `expires_at`, `max_fee_per_cycle`, and `max_priority_fee_per_cycle`. At the end of each block the protocol evaluates due timers against the timer-lane basefee, pre-charges the maximum payable amount from the `fee_payer`, executes the handler, refunds unused gas, and removes the timer. Timers whose `fee_payer` cannot cover the charge, whose fee cap is below the basefee, or whose `expires_at` is reached do not fire.

An **EIP-1559 hybrid** combines the timer-lane basefee and priority tip with a per-actor fairness weight `W(actor) ∈ [1, 2]` over a 1,000-block rolling window. The fairness weight gives under-served actors a bounded priority boost; it does not guarantee execution before expiry.

This design enables sophisticated scheduling strategies: an actor (or a Gas Bidding Agent acting on its behalf) can adjust `max_fee_per_cycle` and `max_priority_fee_per_cycle` per timer based on urgency, network congestion, and balance.

Fairness and Liveness

Timers whose `fee_payer` cannot cover their fire-time charge do not block the lane. Solvent, unexpired timers remain eligible, but a same-height backlog can defer them because the timer lane is finite and independently capped. The per-actor fairness weight is the explicit anti-starvation primitive: actors at or below the network-

median fire rate get the maximum boost, while actors at 2× median or above get none. Because fairness is measured per actor, one controller can split work across several actors; per-fire costs and the lane budget bound that Sybil strategy but do not eliminate it.

DoS Prevention Philosophy

The timer system is a potential vector for denial-of-service attacks. An adversary could attempt to schedule millions of timers at a single block height, overwhelming execution capacity, or fill the timer queue with spam to crowd out legitimate users. Cowboy employs multiple layers of defense:

- **Per-Actor Timer Limits:** Each actor is limited to a maximum of **1,024 active timers** at any time.
- **Non-Refundable Scheduling Cost:** Every scheduled timer pays a flat 200-cycle cost, so failed or cancelled spam is not free.
- **Same-Block Prohibition and Fanout Cap:** A timer cannot fire in the block that creates it, and each transaction may enqueue at most 1,024 messages or timers.
- **Governance-Activated Timer Basefee:** The target EIP-1559 hybrid adjusts a timer-lane basefee with utilisation after Tier-3 activation.
- **Per-Block Execution Budget:** Each block reserves a dedicated portion of its compute budget for timer execution, ensuring timer storms cannot completely crowd out regular transactions.

For complete technical details on these mechanisms, see the Technical Whitepaper.

Protocol-Mediated Off-Chain Compute

Not all computation belongs on-chain. LLM inference, web scraping, and complex data transformations are too expensive for consensus execution, too slow for a 1-second block, and often non-deterministic. Cowboy's answer is a native marketplace of **Runners** — off-chain workers who stake CBY, publish rate cards, and execute jobs under explicit trust profiles.

Job Lifecycle

The marketplace is not a bulletin board that runners scrape; it is a protocol-mediated dispatch pipeline:

1. **Post:** An actor submits a job with an escrowed max price, explicit resource bounds (tokens, wall time, memory), and a chosen trust profile.
2. **Assign:** The protocol samples Runners via stake-weighted VRF from the eligible set — staked, healthy, and whose advertised capabilities satisfy the job's entitlements. The request is bound in the submission block, and every assignment draws from the threshold seed three absolute views later. This adds a minimum three-view dispatch delay, including for single-Runner jobs, and prevents request contents from being chosen after learning that request's seed. It does not prevent paid multi-request sampling or proposer inclusion strategies. Quorum profiles require nine eligible Runners at launch and fail admission rather than silently weakening the threshold.

3. **Commit / Reveal:** Committee members commit to output hashes before revealing their outputs. This prevents a member from changing its reveal after observing another valid reveal; it does not prevent collusion or pre-commit sharing.
4. **Challenge, where supported:** For profiles with objective failure rules, a bonded challenge window can penalize proven violations. The `none` and `economic_bond` profiles do not provide per-result objective slashing.
5. **Payout:** 89% of the job payment goes to the runner(s), 10% is burned, and 1% funds the Treasury. Unused escrow returns to the actor.

Pricing is a free market: `actual_payment = min(reported_usage × rates, max_price)`. The protocol deliberately does **not** meter off-chain execution with gas — deterministic on-chain metering and non-deterministic off-chain resource pricing are different problems, and conflating them (as single-gas-scalar designs must) makes both worse.

Trust Profiles

`M` is the number of assigned Runners; `N` is the number of matching results required for admission. Developers choose the right balance of evidence and cost per job:

MODE	RUNNERS	EVIDENCE ESTABLISHED
`none`	1	Delivery only; no result verification
`economic_bond`	1	Delivery backed by general stake; no objective per-result slashing
`majority_vote`	N-of-M	Agreement on a declared field value
`structured_match`	N-of-M	Agreement under declared verifier functions
`deterministic`	N-of-M	Exact matching, with TEE required by this profile
`semantic_similarity`	N-of-M	Clustering above a declared embedding threshold

TEE requirements can constrain eligible Runners where a profile defines the corresponding attestation checks. ZK result proofs provide a separate profile for computations with a supported proof circuit and verifier.

LLM output is non-deterministic: repeated executions may produce byte-different outputs that a declared verifier considers semantically similar, so byte-exact quorum is not generally suitable. Structured matching, embedding similarity, and economic bonds can provide bounded evidence and economic accountability. They do not establish that a subjective answer is true, that committee members are independent, or that a claimed model produced the answer unless a stronger attestation or proof supplies that fact.

Bring Your Own Compute

The Runner market is designed for open supply: operators stake, register rate cards, declare capabilities, and compete for eligible jobs. This is a load-bearing decision, but it does not remove the selected Runner from the trust boundary:

- **No mandatory platform compute provider.** Selection is protocol-mediated rather than hard-wired to one inference vendor. The selected Runner still receives the job inputs it is authorized to process unless a stronger isolation profile applies.
- **Markets find the hardware.** Idle GPUs, specialized accelerators, region-specific capacity, and consumer hardware all clear at their own prices through rate cards, instead of being flattened into one provider's SKU list.
- **Capability matching replaces gatekeeping.** The entitlements system matches jobs to runners on declared facts — supported models, TEE hardware, geographic region, network egress — so specialization is expressed in the protocol, not negotiated in a sales channel.
- **Run your own supply.** An application with strict requirements can run its *own* Runners. Its compute joins the same market, serves its jobs, and remains subject to the same authorization and settlement rules. This is the marketplace used at N=1, not a separate deployment mode.

Bring Your Own Models

The same logic extends to model weights. Runners advertise which models they serve; nothing requires those models to be public:

- **Monetize without publishing.** A model owner can serve a proprietary model from its own Runners, publish a rate card, and earn per-inference revenue without distributing the weights. The chain records selection, commitments, evidence, and settlement according to the chosen trust profile; it does not inspect or prove the model artifact itself.
- **Private weights as private data.** Weights can live in a private CBFS volume, with the decryption key released through CBSS only to an authorized Runner for a dispatched job. That authorized Runner is able to process the plaintext. Excluding its host operator requires an additional isolation mechanism such as an accepted TEE profile; CBFS plus CBSS alone does not make that claim.
- **Why this matters:** open-weight models commoditize; frontier and fine-tuned models are where economic value concentrates. A platform that can only serve public models cedes that entire economy to centralized APIs. Cowboy's design lets model owners participate in a permissionless market while keeping the asset that makes them valuable.

The Sovereign Substrate

A **near-chain service** is a protocol-defined, stateful resource service with an on-chain lifecycle or authority record and a provider-served data path outside consensus. This document uses the term for CBFS, CBSS, and CBQS; per-job Runner execution is the separate compute plane. Together the three near-chain services form the

sovereign substrate beneath chain-anchored applications. Consensus state is the wrong home for most of what a real application accumulates: it is public by construction, priced for scarcity, and sized for compact state transitions.

> **Consensus records the resource lifecycle, authority, commitments, and settlement each near-chain capability requires. Relays, proxies, and brokers carry storage objects, release shares, and queue records outside consensus; Runners are the adjacent compute plane.**

The split is subsystem-specific rather than absolute. CBFS anchors volume lifecycle and manifests while Relay Nodes store objects. CBSS derives release authorization from chain state while a threshold proxy committee performs key release. CBQS registers streams, providers, pricing, and settlement on-chain while the selected broker remains authoritative for accepted records and consumer progress.

CBFS: Storage

What it is: A decentralized, erasure-coded object store operated by permissionless, staked **Relay Nodes**, mounted by accounts as private (client-side encrypted) or public volumes.

The decisions behind it:

1. **Separate state from storage.** State is consensus-critical key/value data billed per cell and subject to rent. Storage is bulk data — agent memory, embeddings, artifacts, web assets — referenced on-chain only by compact manifest-root commitments. Ethereum fuses these; the fusion is why "put it on-chain" is a joke and "put it on S3" is a surrender.
2. **Off-chain data plane.** Reads and writes are client-to-relay RPCs authenticated by capability tokens; the chain is touched only at lifecycle boundaries (create, anchor, bill, repair, evict). An agent reading its memory thousands of times a day pays for storage, not for consensus.
3. **Private by default, public by choice.** Private volumes are encrypted client-side: relays store and serve ciphertext, and the protocol never asks storage operators to be trusted with private content — so the storage network can be permissionless without being a privacy hazard. Public volumes are deliberately plaintext and world-readable without a CapToken — that is what lets the gateway serve web assets and public datasets straight from relays. A volume's visibility is chosen at creation and immutable thereafter.
4. **Capability tokens over transactions.** Scoped, signed **CapTokens** (volume, path prefix, mode, quota, expiry) grant access — chain-issued for runner mounts at job dispatch, client-signed via a **delegation keypair** for owner access. Cold wallet authority and hot data-path authority are deliberately separated.
5. **Auditable durability.** Reed-Solomon shards across independent relays and on-chain Proof-of-Retrievability challenges produce evidence about storage service. Failed proofs trigger protocol-defined penalties and slashing.

CBSS: Secrets

What it is: Threshold identity-based encryption (BLS12-381, the drand tlock construction) operated by an open, staked proxy committee, releasing secrets to the Runner authorized for a dispatched job. The threshold prevents any one proxy from deriving the plaintext, while registration, staking, and accountability rules govern committee participation.

The decisions behind it:

1. **Conditional plaintext release is the missing primitive.** Chains make everything public; private rollups make everything available to whoever decrypts the rollup; off-chain vaults release plaintext against a bearer token with no link to on-chain reality. Agents need a third thing: *release this secret only to whoever the chain says is doing this job* — so Cowboy built it as infrastructure.
2. **Thresholds, not hardware.** Decryption requires *t*-of-*n* proxy cooperation; the trust root is threshold non-collusion plus the protocol's registration, audit, and accountability records. TEEs are an additional execution profile, not the foundation of secret release.
3. **Chain-derived authorization.** Proxies validate each release request against current chain state: the actor's manifest declares the secret, the job is real, the requesting runner is the one dispatched. Authorization is computed from the authority record, not from possession of a credential.
4. **Actors hold references and policy, not plaintext.** An actor can operate authenticated integrations while its public code and state contain a secret reference and release policy rather than the credential. An authorized Runner crosses into the plaintext boundary when it combines the release shares; applications that need stronger host isolation must request it separately.

CBSS is also the key-delivery layer for private CBFS volumes and provides **time-lock release** (encryption to a future block height), which the protocol reuses for sealed-bid auctions.

CBQS: Coordination

What it is: A protocol for durable streams and queues for off-chain workloads — at-least-once delivery, replay, consumer groups, and push delivery — served by chain-registered broker providers. Stream configurations can encrypt payloads end-to-end; the protocol does not hide topology or all operational metadata.

The decisions behind it:

1. **Mailboxes are not a message bus.** Actor mailboxes carry public, consensus-relevant state transitions at consensus speed and gas cost. Multi-agent applications also need high-frequency, private, durable transport between runner-hosted workloads — a coordinator fronting a chat surface, workers fanning out tasks. Before CBQS, every serious multi-agent build re-implemented persistence, retry, fan-out, cursors, and crash recovery on its own. Repeated reinvention of the same infrastructure is a signal the platform is missing a primitive.
2. **Chain lifecycle, broker records.** A stream is created, owned, configured, and billed through chain state; message flow is broker RPC rather than a chain write per record. The broker is authoritative for accepted records, leases, cursors, and consumer progress. Provider reassignment uses an explicit export and continuity procedure that transfers the authoritative stream state to a replacement broker; re-homing is a defined recovery operation, not an implicit guarantee of seamless migration.
3. **Configured confidentiality, explicit metadata.** Encrypted streams keep content keys away from brokers and the chain, but stream records, release receipts, connection metadata, and traffic patterns remain visible at their respective layers. Self-hosting can reduce exposure to a third-party broker; it does not erase metadata already committed elsewhere.

Why the Iceberg Shape

The control/data split is what makes the sovereign application pole possible. Its properties trace to specific mechanisms:

- **Scoped authority** — each subsystem says exactly which lifecycle, policy, commitment, and settlement facts belong in consensus and which operational facts remain authoritative at a provider.
- **Configured privacy** — private CBFS volumes and encrypted CBQS streams protect content from their storage or broker providers; public volumes and operational metadata are deliberately outside that claim; authorized Runners can see the plaintext they process.
- **Provider choice with explicit procedures** — Runner, Relay, proxy, and broker supply are open to protocol-registered providers; each subsystem defines the evidence and state transfer required to replace a provider.
- **Economic accountability** — rate cards, escrow, receipts, commitments, challenges, and rent connect off-chain work to on-chain payment without pretending every result is reproduced by validators.

An application can compose these guarantees per operation: public consensus for ownership and settlement, private volumes for documents or model weights, threshold release for credentials, encrypted streams for coordination, and a selected Runner trust profile for computation. This is what *sovereign* means in Cowboy: the application retains its chain-anchored identity and authority while using subsystem services around it, with provider replacement following the export, continuity, and recovery rules of each subsystem.

Sovereignty as an Exit Property

A chain record can establish ownership without guaranteeing operational sovereignty. Evaluate each near-chain subsystem against five properties:

1. **Export:** Can the owner retrieve the complete data and metadata needed to continue elsewhere?
2. **Replace:** Can authority move to another provider without the incumbent's cooperation?
3. **Recover:** What survives if the provider disappears, corrupts state, or refuses service?
4. **Revoke:** How quickly do old capabilities, replicas, and release rights become unusable?
5. **Verify:** Which provider claims can the owner or protocol independently check?

Chain-anchored identity and capability control are necessary, but they are not substitutes for data availability, provider migration, or recovery. For a property to count in Cowboy's sovereignty claim, the subsystem specification identifies its authoritative state, export artifact, replacement or recovery procedure, revocation latency, adversary, and independently verifiable evidence.

Dual-Metered Gas

Ethereum introduced **gas** for execution and state access, and EIP-4844 later added an independent blob-gas market for data availability. Cowboy splits its application execution pricing into two independent meters:

- **Cycles** measure compute: Python operations and host calls (e.g., send, set-timer, blob-commit) each have a fixed cost. Cycles resemble **Erlang reductions**: a budget of discrete steps that bounds how long a handler runs.

- **Cells** measure bytes: calldata, return data, blobs, and storage all consume cells.

Each block adjusts two basefees (one per meter) using the familiar EIP-1559 feedback loop. Users specify max prices and optional tips for each meter. Basefees are **burned**, while tips go to validators. This dual model makes fees more predictable and fair.

Why Separate Meters?

Separating compute and data pricing provides several benefits:

1. **Fair Pricing:** A transaction that processes large amounts of data but does little computation pays for data, not computation. Conversely, a compute-intensive transaction pays for cycles, not data.
2. **Predictable Costs:** Developers can reason about costs independently: "This operation will cost X cycles and Y cells" rather than trying to understand how a single gas scalar maps to both dimensions.
3. **Better Resource Management:** The protocol can adjust pricing for compute and storage independently based on network conditions. If storage is scarce but compute is abundant, cell basefees rise while cycle basefees fall.
4. **Independent Basefees:** Each meter has its own EIP-1559-style basefee update and burn rule.

Consensus Philosophy

Cowboy combines **Simplex BFT** with proof of stake, a Cowboy-defined VRF leader schedule, and lane-specific transaction processing. These components have distinct roles.

Why Simplex?

1. **Compact consensus core:** Simplex uses same-view notarize, application-certify, finalize, and nullify phases. This keeps the consensus logic that must be implemented and audited small.
2. **VRF-based leader election:** A leader is elected for each absolute consensus view. The same validator can lead consecutive views, so election does not remove single-proposer discretion or public-mempool MEV.
3. **Fast-finality target:** Cowboy targets roughly two seconds across proposal, notarization, application certification, and finalization phases within the proposal's view. This is a target to validate under realistic geographic latency, not a substitute for measured network evidence.

MEV Resistance Through Design

Cowboy takes a multi-layered approach to MEV mitigation:

- **VRF-Based Transaction Ordering:** Candidate batches are finalized before their ordering seed is revealed, then ordered deterministically within each lane. This reduces discretionary placement inside the committed batch; it adds pipeline latency and does not prevent pre-commitment censorship, paid multi-transaction sampling, or private order flow.

- **Dedicated Lanes:** Block space is partitioned into reserved lanes for system operations, timers, and runner results. This isolates congestion between lanes; it does not prevent an attacker from competing with a victim inside the same lane.
- **Public Mempool:** Cowboy does not require an encrypted mempool. That choice accepts single-block inclusion, private-order-flow, and JIT MEV risks and should be revisited against observed workloads rather than justified by fast finality alone.

For complete consensus specifications, see the Technical Whitepaper.

Economic Model

Cowboy's economic design specifies payments, burns, issuance, jailing, and slashing for validators, Runners, and actors:

Fee Burns

All basefees (for both Cycles and Cells) are **burned**. Burns offset some or all gross issuance depending on network usage; the protocol does not guarantee net deflation, token appreciation, or a particular benefit to holders.

Validator Rewards

- **Block Inflation:** Validators receive rewards from a declining gross-inflation schedule on a fixed genesis supply of 1,000,000,000 CBY: 8%/6% in the bootstrap years, 4%/3% on the glidepath, and a 2% floor at steady state. Gross inflation is offset by protocol burns, so net inflation depends on network usage.
- **Tips:** Proposers receive transaction tips.
- **Conservative Slashing:** Most specified offenses cause temporary jailing; stake is destroyed only for faults assigned a slashing penalty.

Runner Marketplace

Off-chain compute is priced via a **free market**:

- Runners set their own rates via on-chain rate cards
- Jobs are matched to runners based on price, capabilities, and entitlements
- Economic bonds and reputation affect eligibility and consequences for delivery failures and defined faults; they do not prove result correctness
- Each job settlement pays 89% to runners, burns 10%, and sends 1% to the Treasury. These are protocol-configured settlement destinations, not a claim about token value.

State Rent

Persistent storage incurs **ongoing rent**. Unpaid rent first enters a seven-epoch grace period and then a three-epoch warning period. After ten unpaid epochs, storage is evicted and can be restored only if someone retained the original data and supplies it with the required payment and a proof against the recorded storage root hash. Off-chain CBFS storage is billed separately — per MiB per epoch, in its own market — keeping bulk data costs out of the consensus fee markets.

USD-Denominated Billing

Agents and their users should not need to hold a volatile asset to buy inference. Billing can be denominated in **CUSD**, a fiat-backed, transfer-restricted USD credit built on the standard fungible-token transfer hook and settled through the payment layer, while gas remains invisible to the end user. This keeps the commercial product surface in stable units without changing any tokenomic rule: CBY remains the protocol's staking, gas, and burn asset.

Application and Commerce Rails

Beyond actors and provider capabilities, Cowboy defines protocol interfaces for making software addressable, payable, distributable, and composable. Well-known system addresses avoid granting protocol privilege to one store, gateway, or application contract:

RAIL	PROTOCOL FUNCTION
Routing and ingress	The Route Registry and Gateway target actor methods, public CBFS volumes, Runner handlers, and registered persistent workloads. Manifest-driven bindings and the Gateway Registry provide provider discovery and marketplace selection.
Session payments	The Session Actor (**0×0C**) escrows value and settles cumulative vouchers; PaymentGate (**0×12**) applies the same settlement model to metered HTTP sessions.
Actor hiring and distribution	Trading Post (**0×1E**) and store-neutral publication rails define declarations, versions, grants, direct and storefront hires, settlement, review attestations, artifact publication, and sandbox-determinism validation.
Container execution	One-shot container jobs and Container Registry policy (**0×13**) govern images and bounded jobs; persistent workload records, health leases, Gateway routing, Runner provisioning, supervision, rescheduling, and third-party billing govern long-lived services.
Events and feeds	Event Subscription defines the virtual subscription interface and EventListener supplies the system actor endpoint. A feed product such as Watchtower is an application or provider built on these primitives, not a synonym for the system actor.
Intent settlement	Intent Settlement (**0×14**) matches and settles declared intents under the protocol's conservation rules.

Entitlements: Least-Privilege by Default

Cowboy implements a declarative permissions system called **Entitlements** that governs actor and runner capabilities. This system is fundamental to Cowboy's security model.

Entitlements enforce least-privilege by default:

- **Actors declare requirements:** What permissions they need (HTTP domains, TEE access, storage quotas)
- **Runners advertise capabilities:** What they can provide (supported models, geographic regions, TEE hardware)
- **Scheduler enforces matching:** Jobs only run on runners where `requires ⊆ provides`
- **Syscalls are gated:** Operations fail if the actor lacks the corresponding entitlement

This creates an auditable, on-chain permission manifest for every actor. Users can inspect exactly what an actor can do before interacting with it.

For the complete entitlements specification, see the Technical Whitepaper.

Why Cowboy Chooses a Purpose-Built Layer 1

A natural question is why Cowboy operates an independent Layer 1 rather than settling through Ethereum. Both optimistic and validity rollups can define custom execution environments; liquidity providers can offer fast exits from optimistic systems; and a rollup sequencer can implement application-specific scheduling. Cowboy therefore does not rely on an impossibility claim.

The purpose-built L1 choice is motivated by control over the complete execution and block-production contract:

- **Consensus-native scheduling.** Validators execute due timers and system work as part of the state transition, with dedicated budgets that do not depend on a separate sequencer policy.
- **One authority and settlement domain.** Actors, Runner assignments, near-chain resource lifecycles, fees, and governance share one chain-defined state machine instead of crossing a rollup-to-L1 boundary.
- **Independent economics and upgrades.** Cowboy can tune block cadence, lanes, fee markets, validator incentives, and execution semantics without inheriting a rollup framework's upgrade or proving constraints.
- **No proving requirement for launch.** A validity rollup would require a proof system for the PVM and Cowboy's enshrined services; an optimistic rollup would require a complete fault-proof and data-availability design. Those are viable architectures, but they are different projects with different security assumptions.

The costs are equally concrete:

- **Independent security budget:** Cowboy must bootstrap and retain sufficient distributed stake rather than inherit Ethereum settlement security.
- **Bridge and liquidity risk:** Assets are not natively composable with Ethereum, and third-party bridges introduce additional trust and failure modes.
- **Operational burden:** The network owns consensus clients, validator operations, upgrades, incident response, and long-range recovery.

This is the real trade: Cowboy chooses protocol autonomy and a unified actor/service control plane in exchange for bearing its own consensus security and interoperability burden. Ethereum's rollup designs remain credible alternatives, and the architecture should be reevaluated if proof systems, shared sequencing, or settlement designs change that balance.

Ethereum Interoperability

Interoperability is a foundational design goal. The same `secp256k1` key can control both a Cowboy account and an EVM address, letting agents hold ETH and ERC-20s, bridge assets, and sign EIP-1559 transactions under tight policy guards enforced by entitlements. The protocol does **not** ship its own bridge validator set — instead, governance selects third-party bridge infrastructure to carry funds and calldata, and exposes the integration to actors through the `bridge.asset` and `bridge.subscribe_event` entitlements. Cowboy actors can subscribe to Ethereum events to trigger on-chain workflows once an `EventListener` system actor (deferred, address to be assigned) is in place.

This interoperability design recognizes that Cowboy and Ethereum serve complementary roles: Ethereum provides liquidity and security for high-value assets, while Cowboy provides the execution environment for autonomous agents. The chosen bridge enables agents to leverage both ecosystems without forcing the protocol to maintain validator sets it does not control.

Intent-Based Settlement

Above the bridge layer, Cowboy adopts an **intent-based** model for exchange and cross-chain flows. An account signs a declarative token diff — "I give up exactly X of token A; I want at least Y of token B" — and off-chain solvers (Runners) compete to fill it. An on-chain Settlement system actor applies matched intents **atomically, only if the bundle conserves value**: no token can be created by settlement, regardless of what solvers or backends do off-chain.

Two decisions distinguish this design:

- **Sealed-bid solver auctions:** Solver competition can run as a sealed auction using CBSS time-lock encryption — bids are encrypted to a future block height and revealed permissionlessly after it passes. This eliminates last-look and bid-sniping, a fairness property open solver markets do not provide.
- **Pluggable corridor backends:** Cross-chain legs go through a uniform settlement interface with governance-selected backends per corridor — the native bridge, ERC-7683/Open Intents adapters for EVM corridors, or NEAR Intents for BTC and non-EVM reach. No standard is treated as a liquidity source: external liquidity always comes from solvers fronting capital over a rail they trust, and the settlement core depends on none of them.

For complete technical specifications on the bridge and interoperability mechanisms, see the Technical Whitepaper.

Security Philosophy

Cowboy's security model prioritizes simplicity and auditability:

- **Constrained Application Code:** Python syntax can improve review accessibility, while the custom PVM, deterministic standard library, deploy gate, and syscall boundary remain consensus-critical attack surfaces requiring independent audits and differential testing.
- **Explicit Semantics:** The message-passing model makes execution flow explicit and traceable.
- **Economic Accountability:** Staking, jailing, slashing, and fees attach defined consequences to delivery failures and provable protocol faults; they do not establish result correctness or prevent collusion.
- **Least Privilege:** The Entitlements system enforces least-privilege access by default.
- **Curious-Provider Threat Model:** Provider data paths assume a relay, proxy, broker, or Runner host may inspect what it holds. Private CBFS volumes, encrypted CBQS streams, and threshold release reduce specific exposures, but they do not create one universal confidentiality guarantee: public volumes and operational metadata are visible, and an authorized Runner can see the plaintext it processes unless a stronger isolation profile applies.

A public-network security case therefore depends on published validator-concentration and weak-subjectivity assumptions; threshold-key rotation and recovery procedures; client-diversity and correlated-provider-failure analysis; governance-compromise recovery; and measured behavior at the proposed one-second block cadence. The corresponding release gates are specified in Technical Whitepaper §12.6.

For complete security specifications and considerations, see the Technical Whitepaper.

Prior Art and Differentiation

Cowboy combines established ideas in a particular control-plane architecture. Naming the closest systems makes the novel boundary clearer and avoids implying that actors, native timers, storage proofs, or off-chain compute were invented here.

- **Actor-oriented chains:** TON models accounts as actors that process messages and emit outbound messages; the Internet Computer provides canister timers and asynchronous inter-canister execution. Cowboy differs in its Python PVM, dedicated timer lane, and protocol-mediated Runner and near-chain service boundaries.
- **Object and ownership models:** Sui makes object identity and ownership first-class and uses those facts for execution and agreement. Cowboy instead centers durable actor identity and capability-scoped service relationships.
- **Off-chain computation and automation:** Chainlink Functions, Automation, and decentralized oracle networks demonstrate external computation and scheduled triggering. Cowboy enshrines job assignment, settlement, and multiple trust profiles in its own state machine.
- **Storage markets:** Filecoin's Proof-of-Replication and Proof-of-Spacetime are foundational prior art for economically accountable storage. CBFS uses a different relay, erasure-coding, capability, and manifest model with accountable Proof-of-Retrievability challenges and slashing.

- **MEV research:** Flashbots and Ethereum's proposer-builder separation work show that fast finality and leader election do not resolve block-construction incentives. Cowboy's VRF ordering is one bounded mitigation, not a complete MEV design.
- **State lifecycle:** Ethereum's statelessness and state-expiry research frames the same resurrection and historical-data availability problems Cowboy must solve for evicted actors.

Primary sources for these comparisons are listed in Technical Whitepaper §18.

Applications

The two poles map onto the systems above:

- **Consensus applications** — trading agents, DeFi automation, oracle committees, and on-chain games — center deterministic actors, native timers, and protocol-mediated Runner calls, with consequential decisions committed in consensus.
- **Sovereign applications** — durable workspaces, personal AI, multi-agent businesses, and private-model services — are chain-anchored and Runner-centric: identity and authority live in actors, executable logic uses Runner jobs or persistent workloads, and data, credentials, and coordination use the sovereign substrate.

These are not mutually exclusive product categories. A hireable application may publish a versioned actor through the Trading Post, settle usage through protocol payment interfaces, serve public assets through CBFS and a Gateway route, keep private memory in another CBFS volume, and run a persistent backend. Most production systems combine both shapes.

Cowboy's design goal is to make each boundary — replicated state, private data, authorized plaintext, provider execution, evidence, and settlement — explicit and inspectable while preserving chain-anchored identity, ownership, and settlement across the application. A near-chain provider can remain authoritative for operational state, such as CBQS accepted records and consumer progress, without becoming the application's owner.

For complete technical specifications and parameters, see the Technical Whitepaper.